

# Arquiteturas CNN leves para reconhecimento de padrões em LUMA

## Resumo executivo

Para um cenário **sem hardware-alvo, orçamento de latência ou tamanho de dataset definidos**, a escolha mais segura e tecnicamente defensável para reconhecimento de padrões em **LUMA** é **MobileNetV3-Small** como baseline principal, adaptada para **entrada de 1 canal**, seguida por **EfficientNet-Lite0** quando a prioridade for ganhar margem de acurácia, e por **ShuffleNetV2 1.0x** quando a prioridade for **CPU/ARM** e inferência de lote 1 com operadores muito simples. Essa recomendação se sustenta em três pontos: MobileNetV3 entrega um Pareto muito forte de acurácia/latência em CPU móvel e mantém bom comportamento quantizado; EfficientNet-Lite foi explicitamente simplificada para edge com bom suporte a CPU/GPU/TPU e latências muito competitivas no Pixel 4; ShuffleNetV2 foi desenhada a partir de diretrizes de eficiência prática, minimizando custo de acesso à memória e fragmentação, e reporta excelente velocidade em ARM. <sup>1</sup>

Para **entradas em tons de cinza/LUMA**, a literatura mais consistente aparece em imagens médicas: o trabalho clássico de Xie e Richmond mostrou que **pré-treinamento em ImageNet em grayscale** melhora **acurácia e velocidade** ao transferir para radiografias de tórax, e trabalhos posteriores continuam apontando que há uma **mismatch** real entre pré-treinamento RGB genérico e domínios de imagem predominantemente cinza, como raio-X, CT e MRI. Em outras palavras: se o seu pipeline de decisão opera de fato no domínio LUMA, vale a pena tratar o canal único como um **primeiro cidadão** do projeto, em vez de apenas replicar o canal para RGB e esperar que o backbone “resolva”. <sup>2</sup>

A principal conclusão prática é esta: **não existe uma única “melhor arquitetura” universal**, mas existe uma **melhor estratégia padrão**. Comece com **MobileNetV3-Small 1.0** ou **0.75**, substitua o stem de 3 canais por **1 canal**, compare contra um baseline “rápido de implementar” com **replicação de Y para 3 canais**, faça **INT8 PTQ** e, se a perda passar do tolerável, avance para **QAT**. Se a tarefa for muito sensível a textura fina, iluminação e contraste local, teste em paralelo **EfficientNet-Lite0/Lite1**. Se o deployment final tender a CPU/ARM sem acelerador, mantenha **ShuffleNetV2 1.0x** como terceiro candidato obrigatório. <sup>3</sup>

## Definição do problema e construção de dados LUMA

Em visão computacional aplicada, “LUMA” normalmente significa o canal **Y'** de um espaço do tipo Y'CbCr, isto é, um sinal de brilho derivado de RGB **já codificado por gama**, e não a luminância física linear da radiometria. Em HDTV, a recomendação BT.709 define  $Y' = 0.2126 R' + 0.7152 G' + 0.0722 B'$ ; em BT.601, amplamente usada em SD e muito presente em pipelines legados, os coeficientes são **0.299, 0.587, 0.114**. Para modelagem, isso importa porque o seu dataset precisa refletir **o mesmo padrão de extração de Y'** do ambiente real; misturar indiscriminadamente BT.601, BT.709 e grayscale “genérico” costuma introduzir ruído desnecessário no pré-processamento. <sup>4</sup>

O problema técnico aqui é: **aprender decisão a partir de padrão estrutural, textura, contorno, micro-contraste e variação local de intensidade**, mas abrindo mão da informação cromática. Esse regime é especialmente razoável quando a cor é irrelevante, instável ou pouco confiável, como em

inspeção industrial, OCR e ALPR, vídeo comprimido já em YUV, sensores monocromáticos, radiologia e certos pipelines embarcados com restrição de banda e memória. A vantagem é reduzir dependência de cor e, muitas vezes, melhorar robustez a domínio; a desvantagem é perder pistas discriminativas que em algumas classes estão quase inteiras em chroma. <sup>5</sup>

## Datasets típicos para LUMA-only

| Tipo de tarefa                 | Dataset                      | Por que é útil em LUMA                                                                                                                                                       | Observações                                                                   |
|--------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Padrões simples e sanity check | <b>MNIST</b>                 | Nativamente em tons de cinza; excelente para validar pipeline, quantização e overfitting inicial. <sup>6</sup>                                                               | Saturação fácil; não separa bem arquiteturas modernas.                        |
| Imagens médicas leves          | <b>MedMNIST v2</b>           | Coleção padronizada de 12 datasets 2D e 6 3D, em baixa resolução, ótima para experimentação rápida e comparação de arquiteturas compactas. <sup>7</sup>                      | Excelente para busca de hiperparâmetros e ablação em canal único.             |
| Radiologia real                | <b>ChestX-ray14</b>          | Radiografias são, por natureza, grayscale; benchmark multi-rótulo hospitalar com mais de 100 mil imagens. <sup>8</sup>                                                       | Cuidado com desequilíbrio de classes e split por paciente.                    |
| Textura/iluminação             | <b>KTH-TIPS / KTH-TIPS2</b>  | Foi construída explicitamente com variações de <b>iluminação, pose e escala</b> , o que a torna muito apropriada para testar robustez em LUMA. <sup>9</sup>                  | Excelente para medir sensibilidade a contraste e iluminação.                  |
| Texturas em ambiente natural   | <b>DTD</b>                   | Bom benchmark de textura e atributos texturais; pode ser convertido para Y' mantendo estrutura visual útil. <sup>10</sup>                                                    | Útil para comparar perda de cor vs. manutenção de textura.                    |
| Inspeção/anomalia              | <b>MVTec AD / MVTEC AD 2</b> | Dataset clássico de defeitos industriais; útil quando a decisão depende de micro-textura e contraste local. <sup>11</sup>                                                    | Para classificação, pode-se derivar tarefas a partir de categorias/anomalias. |
| Benchmark brasileiro           | <b>UFPR-ALPR</b>             | Base desafiadora de placas capturadas em cenário real, com movimento e iluminação variada; muito relevante para decisão visual em edge no contexto brasileiro. <sup>12</sup> | Embora colorido, a conversão para Y' é natural em OCR/ALPR.                   |

## Como criar um dataset LUMA-only corretamente

Se a fonte original for **vídeo YUV/YCbCr**, o procedimento preferível é **extrair o plano Y' nativo**, em vez de converter um frame reconvertido para RGB e depois de volta para tons de cinza. Se a fonte estiver em RGB, escolha explicitamente o padrão de extração — idealmente **BT.709** para material HD/ importado de pipelines modernos e **BT.601** para material SD/legado — e gere uma imagem de **1 canal** preservando profundidade de bits e faixa dinâmica. <sup>13</sup>

Depois da extração, o ponto mais importante é o **split sem vazamento**. Em vez de dividir aleatoriamente por imagem, divida por **paciente, dispositivo, sequência de vídeo, objeto físico, veículo, peça ou turno de captura**, conforme o domínio. Em ChestX-ray14, por exemplo, o risco de vazamento por paciente é conhecido; em UFPR-ALPR, um split ingênuo por frame pode contaminar treino e teste com o mesmo veículo em condições quase idênticas. <sup>14</sup>

Uma prática muito valiosa é criar, além do conjunto principal, um **conjunto de robustez LUMA**: versões com ruído gaussiano, blur, compressão, brilho/contraste/gama e variações de iluminação. ImageNet-C formalizou justamente a ideia de testar classificadores em **corruptions comuns**, e KTH-TIPS fornece uma base natural para avaliação sob mudança de iluminação. Para sistemas de decisão reais, isso costuma ser mais informativo do que apenas olhar acurácia em um teste “limpo”. <sup>15</sup>

## Métricas e protocolo de avaliação

Para classificação, as métricas básicas continuam sendo **accuracy, precision, recall** e **F1**. Accuracy mede a fração de acertos; precision mede a razão entre verdadeiros positivos e todos os positivos previstos; recall mede a razão entre verdadeiros positivos e todos os positivos reais; F1 é a média harmônica de precision e recall. Em problemas desbalanceados, **balanced accuracy** e, em tarefas binárias/multirrótulo, **ROC-AUC** frequentemente informam melhor do que accuracy simples. <sup>16</sup>

Para deployment, o protocolo mínimo precisa incluir **latência, memória, FLOPs/MAdds, número de parâmetros** e, idealmente, **energia por inferência**. As ferramentas de benchmark do LiteRT/TFLite reportam tempo de inicialização, latência de warmup, latência em regime estacionário e uso de memória; o benchmark deve ser executado com **múltiplas repetições**, aquecimento prévio e configuração de threads conhecida. A interpretação correta é: parâmetros estimam custo de armazenamento; FLOPs aproximam custo aritmético; latência mede custo real de sistema; memória mede viabilidade prática; energia mede custo operacional. <sup>17</sup>

Em LUMA-only, eu recomendo um protocolo em **quatro camadas**. Primeiro, medir desempenho “limpo” no conjunto de teste principal. Segundo, medir **desbalanceamento** com macro-F1 e balanced accuracy. Terceiro, medir **robustez** em conjuntos corrompidos por ruído, blur, brilho, compressão e variação de contraste/iluminação, seguindo a lógica de ImageNet-C. Quarto, medir **eficiência** no runtime final ou em runtimes representativos, porque a literatura mostra que **latência real** pode divergir materialmente de FLOPs, sobretudo em modelos cujo desenho ignora custo de acesso à memória e fragmentação de operadores. <sup>18</sup>

### Checklist mínimo de avaliação

| Dimensão            | O que reportar                                                                                 | Observação prática                                            |
|---------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Qualidade           | Accuracy, macro-F1, recall macro, precision macro, balanced accuracy, ROC-AUC quando aplicável | Use macro métricas se houver classes raras. <sup>19</sup>     |
| Eficiência estática | Params, FLOPs/MAdds, tamanho do modelo                                                         | Compare sempre na mesma resolução de entrada. <sup>20</sup>   |
| Eficiência dinâmica | Latência média, p95, warmup vs. steady-state, memória total                                    | LiteRT benchmark já separa vários desses itens. <sup>17</sup> |

| Dimensão      | O que reportar                                                      | Observação prática                                        |
|---------------|---------------------------------------------------------------------|-----------------------------------------------------------|
| Robustez      | Queda relativa sob ruído, blur, brilho, contraste, JPEG, iluminação | Use severidades fixas e repita por semente. <sup>21</sup> |
| Generalização | Split por paciente/objeto/cena/dispositivo                          | Evita otimismo artificial. <sup>22</sup>                  |

## Panorama das arquiteturas leves

As famílias mais relevantes para o seu caso se distribuem em três grupos. O primeiro grupo é de **CNNs móveis clássicas**: MobileNetV2, MobileNetV3, EfficientNet-Lite, ShuffleNetV2, GhostNet e SqueezeNet. O segundo é de **TinyML/MCU**, onde MCUNet/TinyNAS/TinyEngine se destacam quando a restrição é realmente extrema. O terceiro grupo é de **híbridos CNN-transformer ou convnets modernizadas**, como MobileViT, EfficientFormer e ConvNeXt-Tiny. Para LUMA, o essencial é separar arquiteturas que são “leves no paper” das que são **eficientes no runtime real**. <sup>23</sup>

MobileNetV2 foi a arquitetura que consolidou o bloco com **inverted residuals** e **linear bottlenecks**, com bom equilíbrio entre custo e desempenho. MobileNetV3 foi além, usando **NAS orientada a hardware**, **Squeeze-and-Excitation** e **h-swish**, melhorando a fronteira acurácia-latência em telefones Pixel. EfficientNet-Lite removeu SE e trocou swish por **ReLU6**, justamente para facilitar quantização e deployment em edge. ShuffleNetV2 partiu de um diagnóstico muito prático: **FLOPs sozinhos explicam mal a latência**, e por isso otimizou largura de canais, fragmentação e custo de acesso à memória. GhostNet explorou a redundância de mapas de características por meio de **ghost modules**, mas a própria evolução da linha de trabalho mostrou que o ganho depende bastante do hardware, a ponto de surgirem variantes específicas para CPU e GPU. SqueezeNet foi pioneira em compactação extrema por meio dos “fire modules”, mas hoje tende a perder em acurácia para alternativas móveis mais recentes. <sup>24</sup>

MobileViT e EfficientFormer mostram que híbridos leves podem ser competitivos, mas com perfis diferentes. MobileViT injeta blocos transformer num backbone móvel e melhora bastante a acurácia para a família de parâmetros, porém sua latência em CPU tradicional ainda pode decepcionar. EfficientFormer foi desenhada explicitamente para velocidade em dispositivos móveis e alcançou latências muito baixas em iPhone 12, inclusive superando híbridos anteriores. ConvNeXt-Tiny, por sua vez, é arquiteturalmente elegante e forte em visão geral, mas **não é “leve” no mesmo sentido** de MobileNet/ShuffleNet/EfficientNet-Lite: seus ~29M parâmetros e ~4.5G FLOPs a colocam numa classe bem mais pesada para edge genérico. <sup>25</sup>

## Comparação resumida de arquiteturas

| Família     | Variante representativa | Ideia central                         | Params | FLOPs / MAdds | Latência publicada             | Leitura para LUMA                                                      | Fonte         |
|-------------|-------------------------|---------------------------------------|--------|---------------|--------------------------------|------------------------------------------------------------------------|---------------|
| MobileNetV2 | 1.0×                    | Inverted residual + linear bottleneck | 3.5M   | 0.3G          | iPhone CPU 8.0 ms; A100 5.0 ms | Excelente baseline simples; muito forte para 1 canal com stem adaptado | <sup>26</sup> |

| Família           | Variante representativa | Ideia central                                           | Params | FLOPs / MAdds | Latência publicada                                                               | Leitura para LUMA                                                 | Font |
|-------------------|-------------------------|---------------------------------------------------------|--------|---------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------|------|
| MobileNetV3-Small | 1.0                     | NAS + SE + h-swish                                      | 2.5M   | 56M           | Pixel 1/2/3: 15.8 / 19.4 / 14.4 ms; INT8 Pixel 1: 15.5 ms                        | Melhor “default” quando custo é crítico                           | 27   |
| MobileNetV3-Large | 1.0                     | NAS + SE + h-swish                                      | 5.4M   | 219M          | Pixel 1/2/3: 51 / 61 / 44 ms; INT8 Pixel 1: 44 ms                                | Quando o problema exige mais capacidade, ainda razoável para edge | 27   |
| EfficientNet-Lite | Lite0                   | Compound scaling simplificado; sem SE; ReLU6            | 4.7M   | 407M          | Pixel 4 CPU 12 ms; GPU 9 ms; GPU FP16 6 ms; INT8 CPU 6.5 ms                      | Muito atraente para LUMA se você puder pagar mais FLOPs           | 28   |
| ShuffleNetV2      | 1.0×                    | Channel split/shuffle; desenho orientado a memória      | 2.3M   | 146M          | ARM 24.4 img/s; GPU 341 batch/s, batch=8                                         | Forte candidato para CPU/ ARM puro e batch 1                      | 29   |
| GhostNet          | 1.0×                    | “Ghost features” via operações baratas                  | ~5.2M  | 142M          | Paper base enfatiza speed/ accuracy; variantes posteriores diferenciam CPU e GPU | Bom potencial, mas menos “seguro” sem hardware definido           | 30   |
| SqueezeNet        | 1.1                     | Fire modules com squeeze/ expand                        | 1.24M  | —             | Sem latência móvel padronizada aqui; 2.4× menos computação que 1.0               | Muito compacto, mas hoje costuma perder em acurácia               | 31   |
| TinyML            | MCUNet                  | TinyNAS + TinyEngine co-desenhados                      | —      | —             | Foco em MCU; >70% top-1 em microcontrolador comercial                            | Só entra no top 3 se o alvo virar MCU de fato                     | 32   |
| ConvNeXt          | Tiny                    | ConvNet “modernizada”, kernel grande, LN, stem patchify | 29M    | 4.5G          | Throughput V100 774.7 img/s                                                      | Excelente backbone, mas pesado demais para ser default em edge    | 33   |

| Família          | Variante representativa | Ideia central                           | Params | FLOPs / MAdds | Latência publicada                           | Leitura para LUMA                                                           | Font |
|------------------|-------------------------|-----------------------------------------|--------|---------------|----------------------------------------------|-----------------------------------------------------------------------------|------|
| Híbrido lite     | MobileViT-XS            | Blocos transformer sobre backbone móvel | 2.3M   | 0.7G          | iPhone NPU 7.2 ms; CPU 26.5 ms; A100 11.7 ms | Interessante se contexto global importar mais que custo CPU                 | 34   |
| Transformer-lite | EfficientFormer-L1      | Transformer orientado a latência móvel  | 12.3M  | 1.3G          | iPhone NPU 1.6 ms; CPU 11.5 ms; A100 6.2 ms  | Muito forte se houver NPU/engine moderno; menos “conservador” como baseline | 34   |

As latências acima **não são diretamente comparáveis linha a linha**, porque vêm de protocolos de medição diferentes: Pixel 1/2/3 com um núcleo grande e batch 1 no caso de MobileNetV3; Pixel 4 com 4 threads no caso de EfficientNet-Lite; ARM/GPU com convenções de throughput no caso de ShuffleNetV2; e iPhone 12 / A100 no caso de EfficientFormer/MobileViT. Ainda assim, a tabela é útil para delimitar **ordem de grandeza** e evitar candidatos claramente fora da faixa de edge leve. <sup>20</sup>

## Evidência empírica em grayscale e LUMA

A evidência mais sólida e direta a favor de uma abordagem **grayscale-first** vem da literatura médica. Xie e Richmond mostraram que modelos **pré-treinados em Grayscale ImageNet** foram um ponto de partida melhor que modelos pré-treinados em color ImageNet para classificação de doenças em radiografias de tórax, com melhora tanto em **acurácia** quanto em **velocidade** na etapa de fine-tuning. Esse resultado é altamente relevante para o seu caso porque mostra que, quando o domínio-alvo é essencialmente cinza/LUMA, o casamento entre distribuição de entrada e representação aprendida importa de verdade. <sup>35</sup>

Essa intuição foi reforçada por trabalhos posteriores que criticam o uso indiscriminado de pré-treinamento RGB genérico em imagem médica cinza. O projeto MedNet explicitamente argumenta que ImageNet “não tem imagens grayscale como MRI, CT e X-ray” e propõe uma variante **Gray-MedNet** para reduzir esse descompasso entre fonte e alvo. Mesmo que o seu domínio não seja médico, a lição transferível é clara: em tarefas onde a decisão depende de **estrutura de intensidade** e não de cor, arquiteturas móveis devem ser adaptadas desde a entrada e, idealmente, também na pré-aprendizagem. <sup>36</sup>

No campo de benchmarks, **ChestX-ray14** continua sendo uma referência importante por reunir mais de 100 mil imagens de raio-X com rotulagem multi-rótulo hospitalar; **KTH-TIPS** é muito útil porque foi concebido com variações de iluminação, pose e escala; e **UFPR-ALPR** oferece um benchmark brasileiro realista, com veículos e câmera em movimento e condições de iluminação variadas, tendo sido acompanhado por um sistema ALPR em tempo real que reporta **35 FPS** no trabalho de introdução. Embora UFPR-ALPR não seja um benchmark “LUMA-only” nativo, ela é particularmente valiosa para avaliar quanto do problema é resolvido por **estrutura luminosa**, já que OCR e localização de caracteres dependem fortemente de contraste, bordas e textura fina. <sup>37</sup>

Na prática, portanto, o que os benchmarks sugerem é o seguinte. Em tarefas de **baixo nível semântico** e alta dependência de contorno/textura — como OCR, placa, defeito, radiografia e textura — a perda de cromaticidade costuma ser muito menos grave do que em classificação geral de objetos naturais. Por isso, o ranking entre arquiteturas leves passa a depender menos de “quem tem maior top-1 no ImageNet RGB” e mais de quatro fatores: **eficiência real em CPU/ARM**, **facilidade de quantização**, **capacidade de preservar detalhe local sob ruído/iluminação** e **facilidade de adaptação do stem para 1 canal**. <sup>38</sup>

## Recomendação de arquiteturas e adaptações para LUMA

### Top 3 recomendadas

| Posição   | Arquitetura                       | Quando escolher                                                 | Justificativa resumida                                                                                                                                            |
|-----------|-----------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Principal | <b>MobileNetV3-Small 0.75-1.0</b> | Escolha padrão para hardware indefinido                         | 2.0–2.5M parâmetros, custo baixíssimo, latência móvel muito forte, bom comportamento quantizado e excelente baseline para stem de 1 canal. <sup>39</sup>          |
| Segunda   | <b>EfficientNet-Lite0</b>         | Quando você pode pagar mais FLOPs por melhor margem de acurácia | Foi simplificada explicitamente para edge, com CPU/GPU/TPU-friendly ops; Pixel 4 mostra latência muito competitiva e boa robustez para quantização. <sup>40</sup> |
| Terceira  | <b>ShuffleNetV2 1.0×</b>          | Quando o deployment tender a CPU/ARM puro e batch 1             | Foi projetada por diretrizes práticas de latência real e memória, não só FLOPs; segue excelente opção conservadora em edge clássico. <sup>41</sup>                |

Eu **não** colocaria GhostNet no top 3 padrão quando o hardware é desconhecido. O motivo não é falta de mérito — GhostNet é muito bom em FLOPs/acurácia — e sim o próprio diagnóstico da linha de pesquisa: a continuação “GhostNets on Heterogeneous Devices” precisou propor variantes **CPU-efficient** e **GPU-efficient** separadas, o que indica que o backbone original é mais sensível ao backend do que um baseline universal deveria ser. <sup>42</sup>

Eu também **não** recomendaria ConvNeXt-Tiny como ponto de partida principal. Ela é excelente como backbone geral, mas seus **29M parâmetros** e **4.5G FLOPs** a colocam muito acima do envelope típico de edge leve; para LUMA-only e decisão embarcada, esse orçamento costuma ser desnecessário, especialmente sem evidência de que o ganho trazido por um backbone tão grande compensará a perda de simplicidade operacional. <sup>33</sup>

Se, no futuro, você descobrir que o deployment real terá **NPU móvel moderna** ou forte aceleração vendor-specific, o candidato “fora do pódio” que mais vale testar é **EfficientFormer-L1**, porque a latência reportada em iPhone 12 é impressionante. Eu só não o elejo como baseline principal agora porque o problema descrito é de **LUMA-only**, com hardware indefinido e foco em decisão prática; nesses cenários, uma CNN móvel madura continua sendo a opção mais robusta em engenharia. <sup>34</sup>

## Modificações recomendadas para LUMA

A modificação mais importante é a **adaptação do stem de entrada para 1 canal**. Em vez de receber RGB, a primeira convolução passa a receber **Y'** diretamente. Isso reduz redundância logo na entrada e força o backbone a aprender filtros iniciais coerentes com contorno, textura e micro-contraste do domínio. Se você precisar de uma baseline imediata com pesos pré-treinados RGB, o atalho mais simples é **replicar Y' em 3 canais**; isso preserva compatibilidade com pesos existentes, mas não é a solução mais elegante nem a mais eficiente. O próprio ecossistema de modelos pré-treinados costuma assumir 3 canais RGB normalizados, como indicado para GhostNet e a maior parte dos backbones de classificação pública. <sup>43</sup>

Para compressão e deployment, a ordem recomendada é: **float32 baseline → PTQ INT8 → QAT INT8**, medindo a perda de acurácia a cada etapa. A documentação do LiteRT/TFLite enfatiza que a quantização pós-treinamento pode reduzir tamanho de modelo e melhorar latência em CPU e aceleradores com pouca degradação; o próprio artigo do MobileNetV3 mostrou resultados quantizados competitivos em Pixel. Em setups realmente restritos, depois de estabilizar QAT, vale avançar para **pruning estruturado** dos canais de expansão e **knowledge distillation**, que o próprio paper do MobileNetV3 já trata como técnica complementar relevante para gerar alunos menores e mais precisos. <sup>44</sup>

Em termos de pré-processamento, eu sugiro tratar LUMA como um domínio de **fotometria local**. Na prática: normalize por média/desvio do dataset, teste **CLAHE** ou equalização local apenas se o domínio sofre com baixo contraste crônico, e use augmentations que reflitam o mundo real do canal Y' — variações de brilho, contraste, gama, blur, ruído e compressão. Isso se alinha diretamente ao tipo de robustez capturado por ImageNet-C e por datasets de textura/iluminação como KTH-TIPS. <sup>21</sup>

## Trade-offs esperados

| Escolha                                        | Ganho esperado                                                                | Perda esperada                                                           |
|------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Stem 1-canal                                   | Menor redundância de entrada; filtros iniciais mais alinhados ao domínio LUMA | Reaproveitamento direto de pesos RGB fica menos trivial                  |
| Replicação Y→RGB                               | Baseline rápida com pesos ImageNet existentes                                 | Desperdício de banda/entrada e menor “alinhamento semântico” ao problema |
| PTQ INT8                                       | Redução de tamanho e latência com baixo custo de engenharia                   | Pode degradar mais que o aceitável em tarefas sensíveis a contraste fino |
| QAT INT8                                       | Recupera parte da perda do PTQ e estabiliza deployment                        | Custo extra de treino                                                    |
| EfficientNet-Lite0 em vez de MobileNetV3-Small | Mais margem de acurácia                                                       | Mais FLOPs e footprint                                                   |
| ShuffleNetV2 em vez de EfficientNet-Lite0      | Melhor previsibilidade em CPU/ARM                                             | Menor teto de acurácia                                                   |
| EfficientFormer-L1                             | Excelente em NPU/engines modernas                                             | Mais dependência do backend e stack de deployment                        |

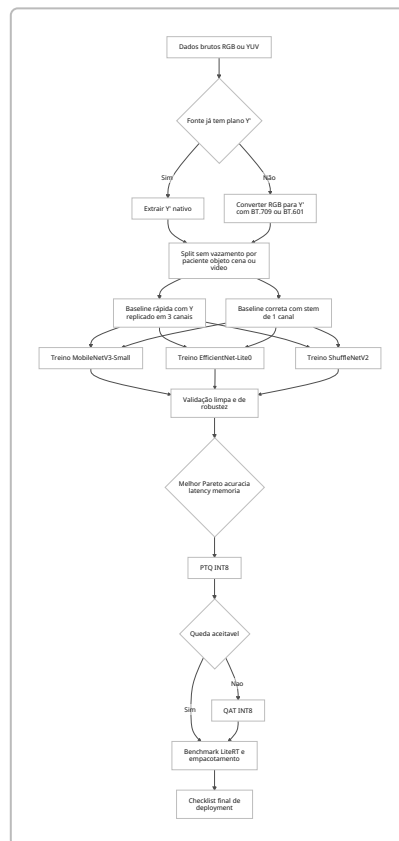


## Plano de implementação e checklist de deployment

Para um primeiro ciclo rigoroso, eu faria **três experimentos-base** em paralelo: **MobileNetV3-Small 1.0**, **EfficientNet-Lite0** e **ShuffleNetV2 1.0x**, todos com resolução entre **160 e 224**, em duas variantes de entrada: **Y' replicado em 3 canais** e **stem de 1 canal**. Isso separa, desde o começo, o efeito da arquitetura do efeito da adaptação de entrada. Se o dataset for pequeno ou moderado, congelar as primeiras camadas por poucas épocas e depois liberar fine-tuning completo costuma reduzir instabilidade inicial; se o dataset for grande e nativamente cinza, vá diretamente para fine-tuning total. A ideia é medir logo de início qual parte do ganho vem de “ser LUMA” e qual parte vem de “ser uma arquitetura melhor”. 45

Como protocolo de treino unificado, um bom padrão moderno é: **AdamW**, **300 épocas** como teto, **warmup** curto, **cosine decay**, early stopping por macro-F1/val loss, e **loss ponderada** quando houver desequilíbrio. Essa receita é próxima da usada por EfficientFormer em ImageNet-1K e é suficientemente estável para comparação entre backbones. Se quiser simplificar ainda mais o estudo inicial, mantenha batch size e augmentations iguais entre os modelos e varie apenas a arquitetura, o stem e a quantização. 34

As augmentations para LUMA devem refletir o que um sistema no mundo real vai sofrer: **RandomResizedCrop** ou crop fixo, flip horizontal quando semanticamente permitido, pequenas rotações/afinidades, jitter de brilho/contraste/gama, ruído gaussiano, motion blur, gaussian blur e compressão JPEG. Para inspeção industrial e OCR, eu incluiria explicitamente **sweep fotométrico** e **sweep de blur** na validação. Para dados médicos, eu seria mais conservador com rotações e deformações. Em todos os casos, crie um pequeno conjunto secundário “corrupto” inspirado em ImageNet-C para reportar queda relativa de desempenho. 15



## Checklist final

| Etapa         | Critério de aceite                                                                             |
|---------------|------------------------------------------------------------------------------------------------|
| Extração LUMA | Fórmula de Y' fixada e documentada; sem mistura silenciosa de BT.601/709                       |
| Splits        | Separação por entidade física/paciente/cena/vídeo, não apenas por imagem                       |
| Qualidade     | Reportar accuracy, macro-F1, recall macro e balanced accuracy; ROC-AUC se binário/multirrótulo |
| Robustez      | Queda relativa medida em blur, ruído, brilho, contraste, compressão e iluminação               |
| Eficiência    | Params, FLOPs, tamanho do modelo, latência média e p95, memória total                          |
| Quantização   | PTQ testado; QAT acionado se a perda for maior que o limiar do projeto                         |
| Deployment    | Benchmark em runtime representativo com warmup e múltiplas execuções                           |
| Decisão       | Thresholds calibrados no conjunto de validação, não no teste                                   |

Os itens de benchmark de latência/memória devem idealmente ser executados com a ferramenta de benchmark do LiteRT/TFLite, que já mede warmup, steady state e memória. Se o alvo final acabar sendo MCU, vale migrar o estudo para um regime TinyML com MCUNet; se o alvo for NPU móvel, introduza EfficientFormer-L1 como quarto candidato na comparação final. <sup>46</sup>

## Fontes primárias, repositórios e datasets sugeridos

### Artigos e código

| Recurso                                    | Referência    |
|--------------------------------------------|---------------|
| MobileNetV2 paper                          | <sup>47</sup> |
| MobileNetV3 paper                          | <sup>48</sup> |
| EfficientNet-Lite model card / checkpoints | <sup>28</sup> |
| ShuffleNetV2 paper                         | <sup>49</sup> |
| GhostNet paper                             | <sup>50</sup> |
| GhostNet código oficial / hub              | <sup>51</sup> |
| SqueezeNet paper                           | <sup>52</sup> |
| SqueezeNet código oficial                  | <sup>53</sup> |
| MCUNet paper                               | <sup>54</sup> |
| MCUNet código oficial                      | <sup>55</sup> |
| ConvNeXt paper / código                    | <sup>56</sup> |
| MobileViT paper / código                   | <sup>57</sup> |
| EfficientFormer paper / código             | <sup>58</sup> |

## Datasets sugeridos

| Dataset              | Uso recomendado                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| MNIST                | Sanity check, prototipagem ultra-rápida em grayscale. <sup>6</sup>                         |
| MedMNIST v2          | Benchmark leve para classificação médica em tons de cinza e baixa resolução. <sup>59</sup> |
| ChestX-ray14         | Benchmark multi-rótulo realista para grayscale. <sup>8</sup>                               |
| KTH-TIPS / KTH-TIPS2 | Robustez a iluminação, pose e escala em textura. <sup>9</sup>                              |
| DTD                  | Texturas “wild” convertidas para Y’. <sup>10</sup>                                         |
| MVTec AD / AD 2      | Defeitos industriais e cenários de inspeção. <sup>11</sup>                                 |
| UFPR-ALPR            | Benchmark brasileiro para decisão visual em tráfego/placas. <sup>12</sup>                  |

Em síntese: para **reconhecimento de padrões em LUMA visando decisão prática**, eu começaria por **MobileNetV3-Small 1-canal**, validaria contra **EfficientNet-Lite0** e **ShuffleNetV2 1.0x**, mediria robustez fotométrica explicitamente, e só depois expandiria para GhostNet ou híbridos transformer-lite conforme o hardware real ficasse claro. Esse caminho maximiza rigor experimental e minimiza o risco de escolher um backbone “bonito no paper” mas ruim no deployment. <sup>60</sup>

<sup>1</sup> <sup>3</sup> <sup>20</sup> <sup>27</sup> <sup>39</sup> <sup>60</sup> [arxiv.org](https://arxiv.org/)

<https://arxiv.org/pdf/1905.02244>

<sup>2</sup> <sup>35</sup> <sup>38</sup> <sup>45</sup> [Pre-training on Grayscale ImageNet Improves Medical Image ...](#)

[https://openaccess.thecvf.com/content\\_eccv\\_2018\\_workshops/w33/html/Xie\\_Pre-training\\_on\\_Grayscale\\_ImageNet\\_Improves\\_Medical\\_Image\\_Classification\\_ECCVW\\_2018\\_paper.html?utm\\_source=chatgpt.com](https://openaccess.thecvf.com/content_eccv_2018_workshops/w33/html/Xie_Pre-training_on_Grayscale_ImageNet_Improves_Medical_Image_Classification_ECCVW_2018_paper.html?utm_source=chatgpt.com)

<sup>4</sup> <sup>13</sup> [Parameter values for the HDTV standards for production ...](#)

[https://www.itu.int/dms\\_pubrec/itu-r/rec/bt/r-rec-bt.709-6-201506-i%21%21pdf-e.pdf?utm\\_source=chatgpt.com](https://www.itu.int/dms_pubrec/itu-r/rec/bt/r-rec-bt.709-6-201506-i%21%21pdf-e.pdf?utm_source=chatgpt.com)

<sup>5</sup> <sup>11</sup> [Industrial anomaly detection benchmark dataset](#)

[https://www.mvtec.com/research-teaching/datasets/mvtec-ad?utm\\_source=chatgpt.com](https://www.mvtec.com/research-teaching/datasets/mvtec-ad?utm_source=chatgpt.com)

<sup>6</sup> [Index of /exdb/mnist](#)

[https://yann.lecun.com/exdb/mnist/?utm\\_source=chatgpt.com](https://yann.lecun.com/exdb/mnist/?utm_source=chatgpt.com)

<sup>7</sup> <sup>59</sup> [MedMNIST v2 - A large-scale lightweight benchmark for 2D ...](#)

[https://www.nature.com/articles/s41597-022-01721-8?utm\\_source=chatgpt.com](https://www.nature.com/articles/s41597-022-01721-8?utm_source=chatgpt.com)

<sup>8</sup> <sup>37</sup> [NIH Chest X-ray dataset | Cloud Healthcare API](#)

[https://docs.cloud.google.com/healthcare-api/docs/resources/public-datasets/nih-chest?utm\\_source=chatgpt.com](https://docs.cloud.google.com/healthcare-api/docs/resources/public-datasets/nih-chest?utm_source=chatgpt.com)

<sup>9</sup> [KTH-TIPS image databases homepage](#)

[https://www.csc.kth.se/cvap/databases/kth-tips/index.html?utm\\_source=chatgpt.com](https://www.csc.kth.se/cvap/databases/kth-tips/index.html?utm_source=chatgpt.com)

<sup>10</sup> [\[1311.3618\] Describing Textures in the Wild](#)

[https://arxiv.org/abs/1311.3618?utm\\_source=chatgpt.com](https://arxiv.org/abs/1311.3618?utm_source=chatgpt.com)

<sup>12</sup> [UFPR-ALPR Dataset - Laboratório Visão Robótica e Imagem](#)

[https://web.inf.ufpr.br/vri/databases/ufpr-alpr/?utm\\_source=chatgpt.com](https://web.inf.ufpr.br/vri/databases/ufpr-alpr/?utm_source=chatgpt.com)

- 14 22 **Comparison of Deep Learning Approaches for Multi-Label ...**  
[https://pmc.ncbi.nlm.nih.gov/articles/PMC6476887/?utm\\_source=chatgpt.com](https://pmc.ncbi.nlm.nih.gov/articles/PMC6476887/?utm_source=chatgpt.com)
- 15 18 21 **Benchmarking Neural Network Robustness to Common Corruptions and Perturbations**  
[https://arxiv.org/abs/1903.12261?utm\\_source=chatgpt.com](https://arxiv.org/abs/1903.12261?utm_source=chatgpt.com)
- 16 19 **accuracy\_score — scikit-learn 1.9.0 documentation**  
[https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy\\_score.html?utm\\_source=chatgpt.com](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html?utm_source=chatgpt.com)
- 17 46 **Benchmark Interpreter API | Google AI Edge**  
[https://developers.google.com/edge/litert/models/measurement?utm\\_source=chatgpt.com](https://developers.google.com/edge/litert/models/measurement?utm_source=chatgpt.com)
- 23 24 47 **MobileNetV2: Inverted Residuals and Linear Bottlenecks**  
[https://arxiv.org/abs/1801.04381?utm\\_source=chatgpt.com](https://arxiv.org/abs/1801.04381?utm_source=chatgpt.com)
- 25 57 **MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer - Apple Machine Learning Research**  
<https://machinelearning.apple.com/research/vision-transformer>
- 26 34 58 **papers.neurips.cc**  
[https://papers.neurips.cc/paper\\_files/paper/2022/file/5452ad8ee6ea6e7dc41db1cbd31ba0b8-Paper-Conference.pdf](https://papers.neurips.cc/paper_files/paper/2022/file/5452ad8ee6ea6e7dc41db1cbd31ba0b8-Paper-Conference.pdf)
- 28 40 **tpu/models/official/efficientnet/lite/README.md at master · tensorflow/tpu · GitHub**  
<https://github.com/tensorflow/tpu/blob/master/models/official/efficientnet/lite/README.md>
- 29 41 49 **ecva.net**  
[https://www.ecva.net/papers/eccv\\_2018/papers\\_ECCV/papers/Ningning\\_Light-weight\\_CNN\\_Architecture\\_ECCV\\_2018\\_paper.pdf](https://www.ecva.net/papers/eccv_2018/papers_ECCV/papers/Ningning_Light-weight_CNN_Architecture_ECCV_2018_paper.pdf)
- 30 43 **GhostNet – PyTorch**  
[https://pytorch.org/hub/pytorch\\_vision\\_ghostnet/](https://pytorch.org/hub/pytorch_vision_ghostnet/)
- 31 52 **[1602.07360] SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size**  
<https://arxiv.org/abs/1602.07360>
- 32 54 **proceedings.neurips.cc**  
<https://proceedings.neurips.cc/paper/2020/file/86c51678350f656dcc7f490a43946ee5-Paper.pdf>
- 33 **arxiv.org**  
<https://arxiv.org/pdf/2201.03545>
- 36 **MedNet: Pre-trained Convolutional Neural Network Model for the Medical Imaging Tasks**  
[https://arxiv.org/abs/2110.06512?utm\\_source=chatgpt.com](https://arxiv.org/abs/2110.06512?utm_source=chatgpt.com)
- 42 **GhostNets on Heterogeneous Devices via Cheap Operations**  
[https://arxiv.org/abs/2201.03297?utm\\_source=chatgpt.com](https://arxiv.org/abs/2201.03297?utm_source=chatgpt.com)
- 44 **Post-training quantization | Google AI Edge**  
[https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post\\_training\\_quantization?utm\\_source=chatgpt.com](https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization?utm_source=chatgpt.com)
- 48 **[1905.02244] Searching for MobileNetV3**  
<https://arxiv.org/abs/1905.02244>
- 50 **GhostNet: More Features from Cheap Operations**  
[https://arxiv.org/abs/1911.11907?utm\\_source=chatgpt.com](https://arxiv.org/abs/1911.11907?utm_source=chatgpt.com)
- 51 **huawei-noah/Efficient-AI-Backbones**  
[https://github.com/huawei-noah/Efficient-AI-Backbones?utm\\_source=chatgpt.com](https://github.com/huawei-noah/Efficient-AI-Backbones?utm_source=chatgpt.com)

53 SqueezeNet: AlexNet-level accuracy with 50x fewer ...

[https://github.com/forrestli/SqueezeNet?utm\\_source=chatgpt.com](https://github.com/forrestli/SqueezeNet?utm_source=chatgpt.com)

55 mit-han-lab/mcunet - Tiny Deep Learning on IoT Devices

[https://github.com/mit-han-lab/mcunet?utm\\_source=chatgpt.com](https://github.com/mit-han-lab/mcunet?utm_source=chatgpt.com)

56 [2201.03545] A ConvNet for the 2020s

<https://arxiv.org/abs/2201.03545>